SupplyPrescript

AI-Powered Supply Chain Decision Support System
Final Project Report

Domain	Supply Chain Operations / Predictive Analytics
Tech Stack	Python · pandas · scikit-learn · XGBoost · FastAPI · HTML/CSS/JS
Dataset	SCMS Delivery History Dataset (~10,324 shipment records)
Development Period	25-day structured build plan
Status	Complete — Days 1–25

Table of Contents

1. Executive Summary

SupplyPrescript is an end-to-end machine learning system that predicts whether a medical supply shipment will be delivered late, and recommends mitigation actions when a delay is likely. The project was completed over a 25-day structured build, covering data preparation, model training, a live backend API, and a tested web dashboard.

The final tuned XGBoost model achieves 89.6% accuracy, 0.579 precision, and 0.342 recall on held-out test data — a substantial improvement over a logistic regression baseline, which never correctly identified a single delayed shipment despite 88.5% accuracy. This gap illustrates the core challenge of the dataset: only ~11.5% of shipments are delayed, so naive accuracy is misleading and recall/precision are the metrics that matter.

The system was built and verified as a working application, not only as notebooks: a FastAPI backend serves live predictions through six tested endpoints, and a web dashboard collects real shipment details and returns input-sensitive predictions with dynamically calculated recommended actions. Edge cases — invalid input, an offline backend, unknown categories — were deliberately tested and handled gracefully.

Known gaps against the original full project vision are documented transparently rather than overstated, most notably that the "recommended actions" use a simplified, transparent cost multiplier rather than a full constrained optimization engine, and that no closed-loop learning (feeding real outcomes back into the model) has yet been implemented.

2. Project Overview

SupplyPrescript predicts whether a shipment will be delivered late (**Is Delayed**), using features known at or before scheduling — vendor, shipment mode, product type, cost, weight, and scheduling date. The system has three layers: a machine learning pipeline that cleans data and trains a classifier, a FastAPI backend serving live predictions, and a web dashboard where a user enters shipment details and receives a prediction plus recommended mitigation actions.

3. Exploratory Data Analysis

Numeric columns were highly right-skewed, with most shipments clustered at low cost/quantity and a long tail of large outliers — this shaped later decisions on feature engineering and evaluation (using metrics robust to imbalance rather than raw accuracy).

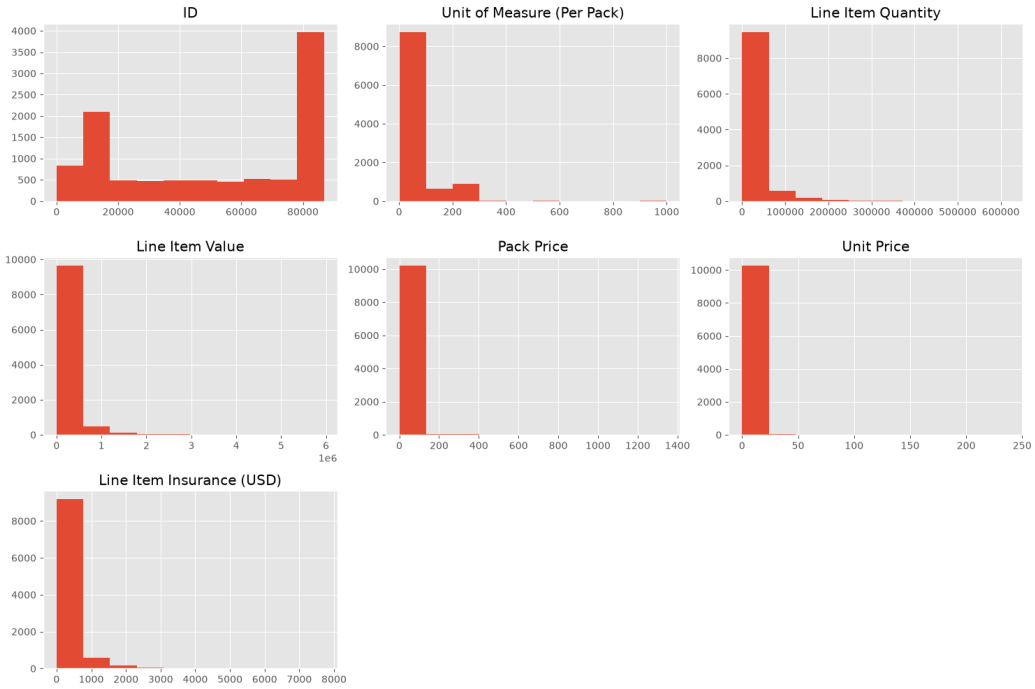

Figure 1 — Distribution of numeric columns (Days 3–5, EDA)

4. Model Training & Results

Data was cleaned, feature-engineered (final shape 10,324 × 158 after one-hot encoding), and split 80/20 with stratification to preserve the ~11.5% delay rate. A Logistic Regression baseline and an XGBoost classifier (tuned via RandomizedSearchCV, optimizing for F1) were trained and compared.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (Logistic Regression)	0.885	0.000	0.000	0.000	0.658
XGBoost (Untuned)	0.895	0.579	0.308	0.402	0.903
XGBoost (Tuned, F1)	0.896	0.579	0.342	0.430	0.882

Key finding: raw accuracy is misleading here due to class imbalance — the baseline model never once predicted a delay despite 88.5% accuracy. XGBoost substantially improves delay detection. Tuning for F1 improved F1 slightly but reduced ROC-AUC, showing the tuning objective directly trades off against ranking quality.

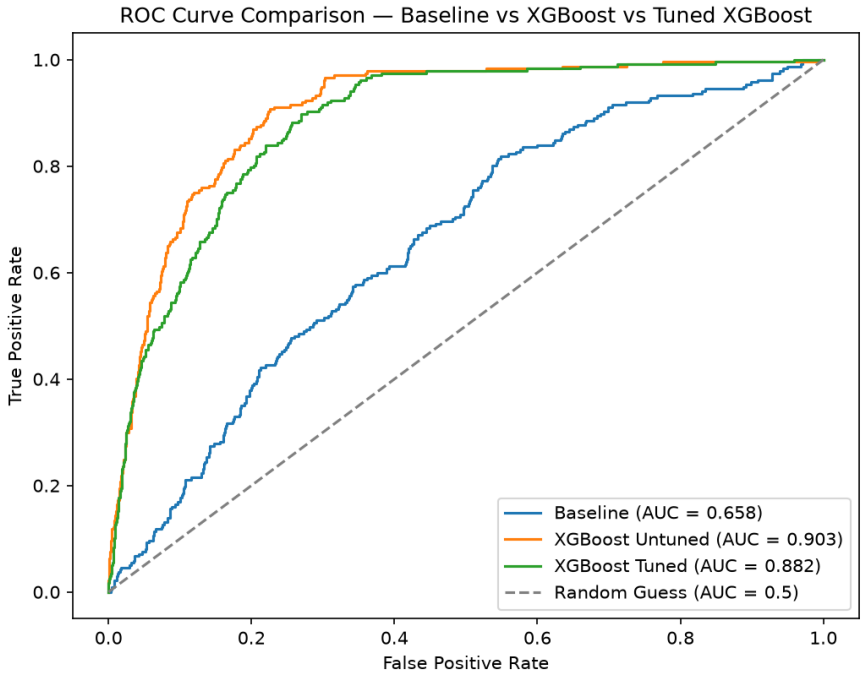

Figure 2 — ROC Curve comparison: Baseline vs. Untuned vs. Tuned XGBoost (Day 12)

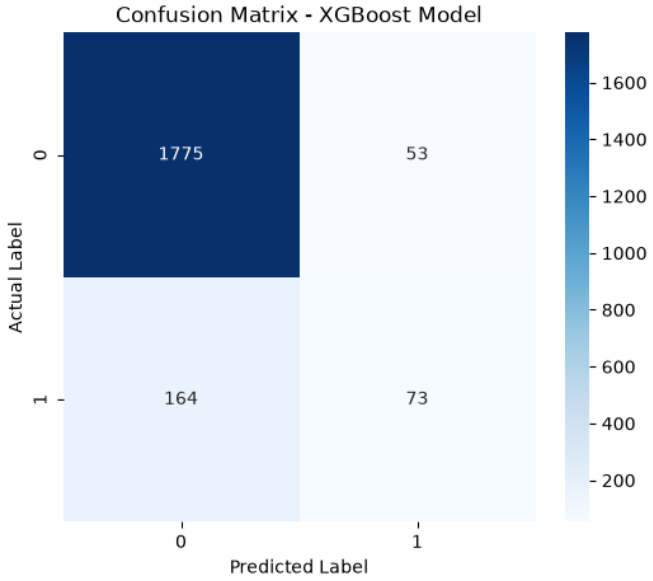

Figure 3 — Confusion matrix, tuned XGBoost. Of 237 actual delayed shipments, 73 were correctly caught and 164 were missed — consistent with the 0.34 recall figure.

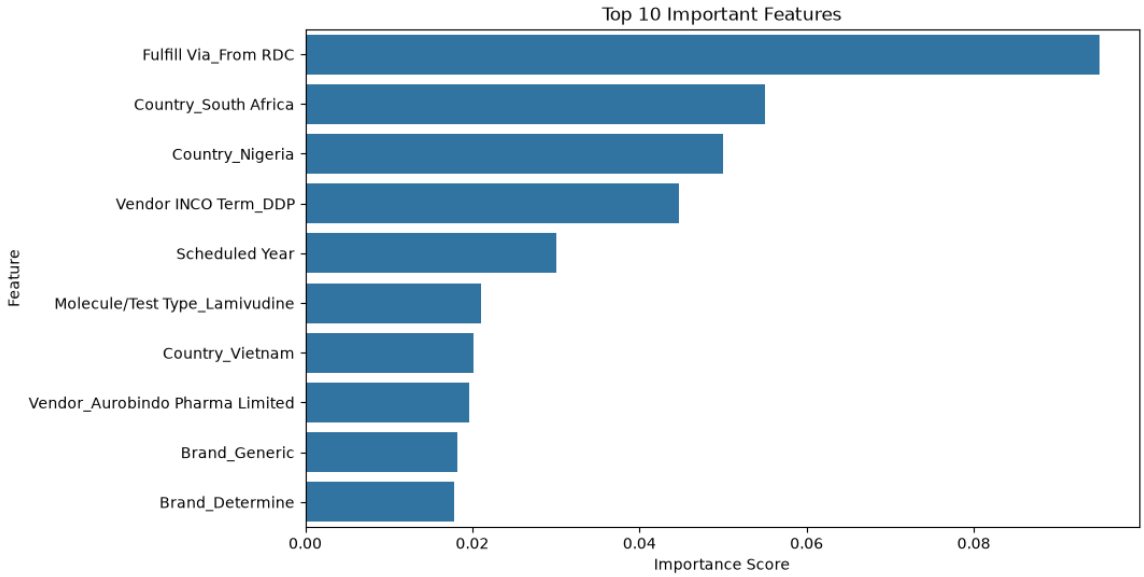

Figure 4 — Top 10 important features, initial XGBoost model (Day 10)

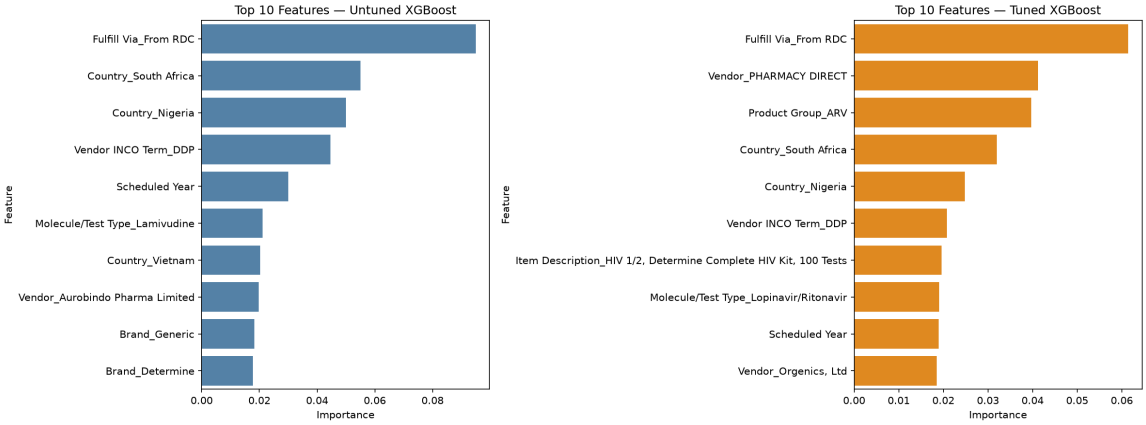

Figure 5 — Feature importance, untuned vs. tuned XGBoost (Day 13). Core drivers — Fulfill Via_From RDC, Country_South Africa, Country_Nigeria, Vendor INCO Term_DDP, Scheduled Year — remain stable across both models, indicating real signal rather than tuning artifacts.

5. Backend API

Built with FastAPI, CORS restricted to the frontend's origin. Six endpoints were implemented and verified live through the Swagger UI.

- **GET /** — health check.
- **POST /predict** — raw endpoint, expects all 157 model columns as a dict.
- **POST /predict_custom** — user-facing endpoint; takes country, shipmentMode, weight, freightCost, and fills the remaining columns with training-data defaults via build_feature_row().
- **GET /test** — fixed demo endpoint, always predicts on row 0 of x_test.csv.
- **GET /columns** — returns the full list of 157 feature columns.

- **GET /compare** — debug endpoint comparing model vs. dataset columns.

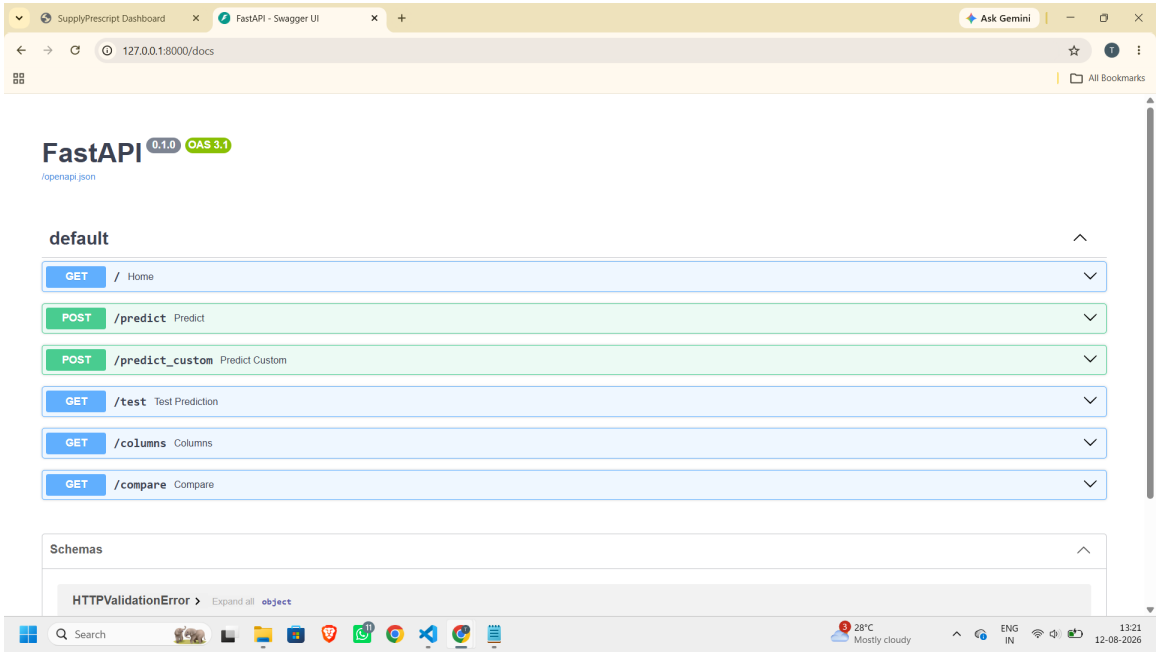

Figure 6 — Live Swagger UI (/docs) showing all six working endpoints

6. Frontend Dashboard — Live Demo Evidence

The dashboard collects real shipment details and calls the backend live. Below is the full tested flow: initial state, a genuine "on time" prediction, a genuine "delay expected" prediction (with dynamically calculated recommended actions), input validation, and graceful handling when the backend is offline.

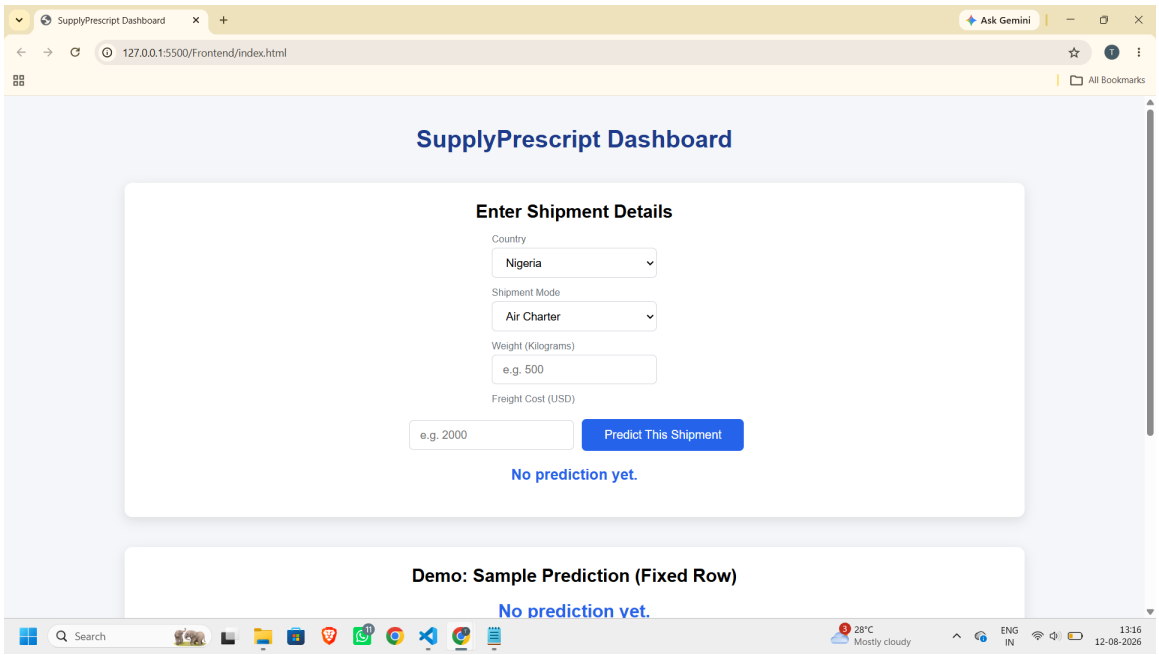

Figure 7 — Dashboard initial state

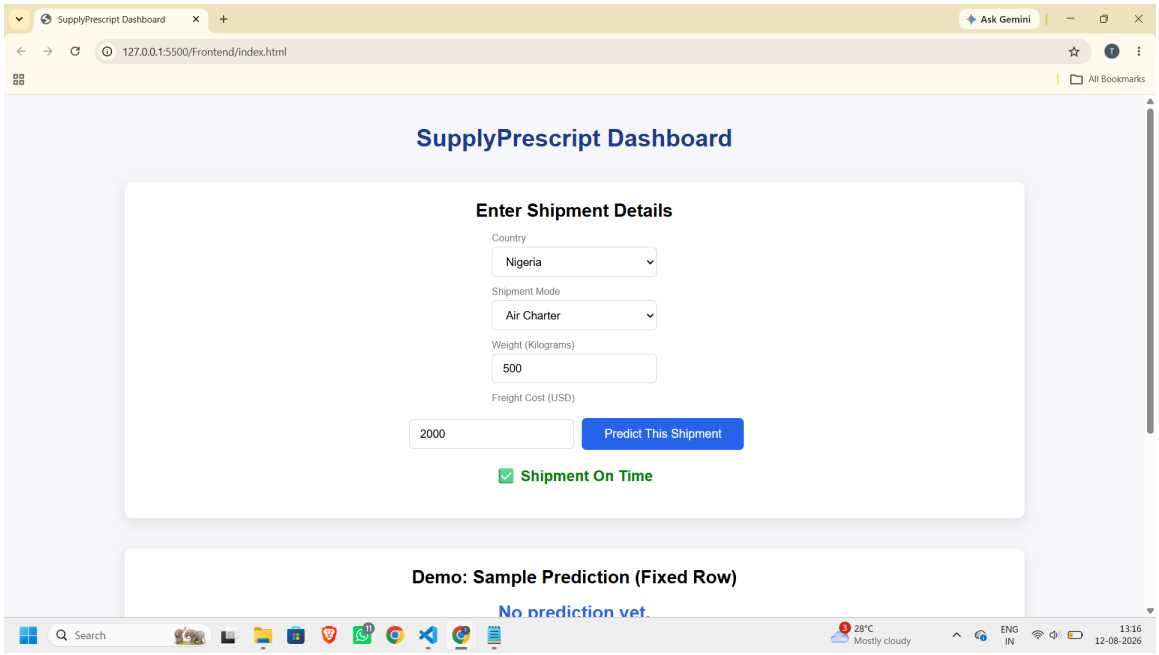

Figure 8 — Live prediction: Shipment On Time (Nigeria, Air Charter, 500kg, $2,000)

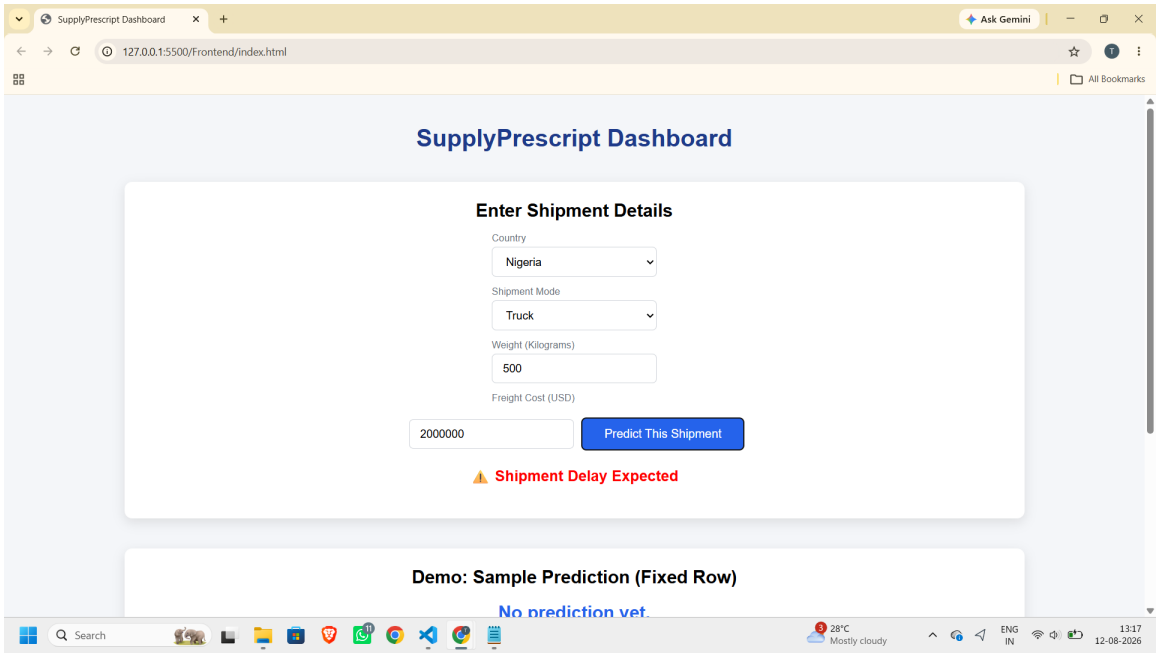

Figure 9 — Live prediction: Shipment Delay Expected (Nigeria, Truck, 500kg, $2,000,000)

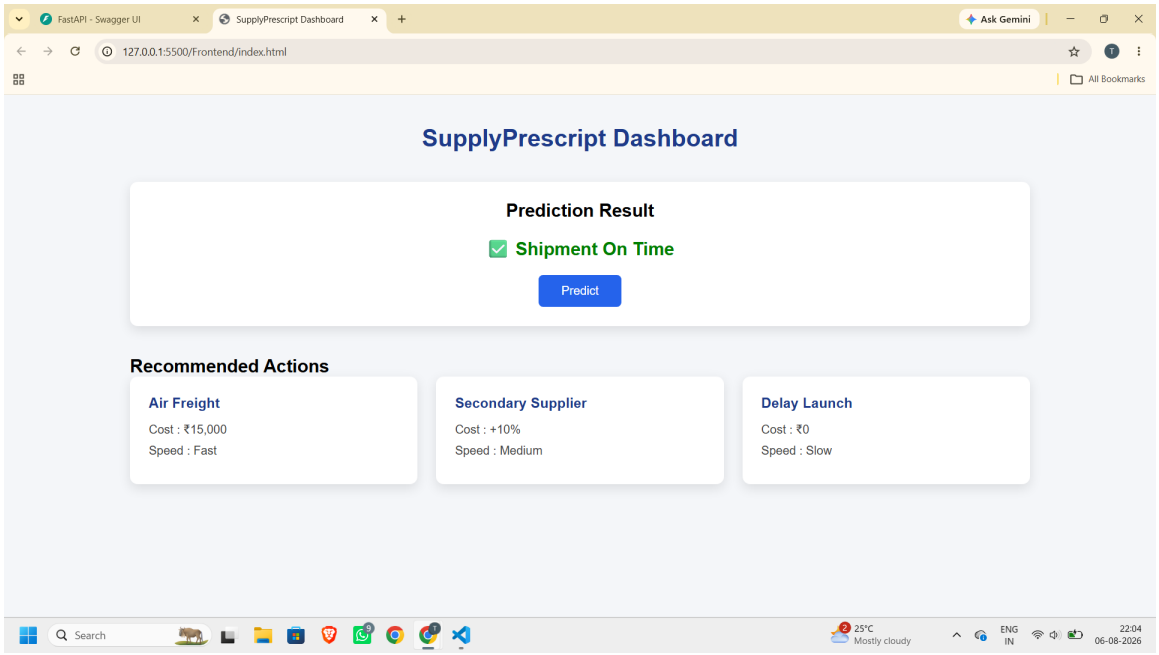

Figure 10 — Full dashboard with dynamically calculated Recommended Actions shown only on a delay prediction (Air Freight = 1.5× entered freight cost, Secondary Supplier = +10%)

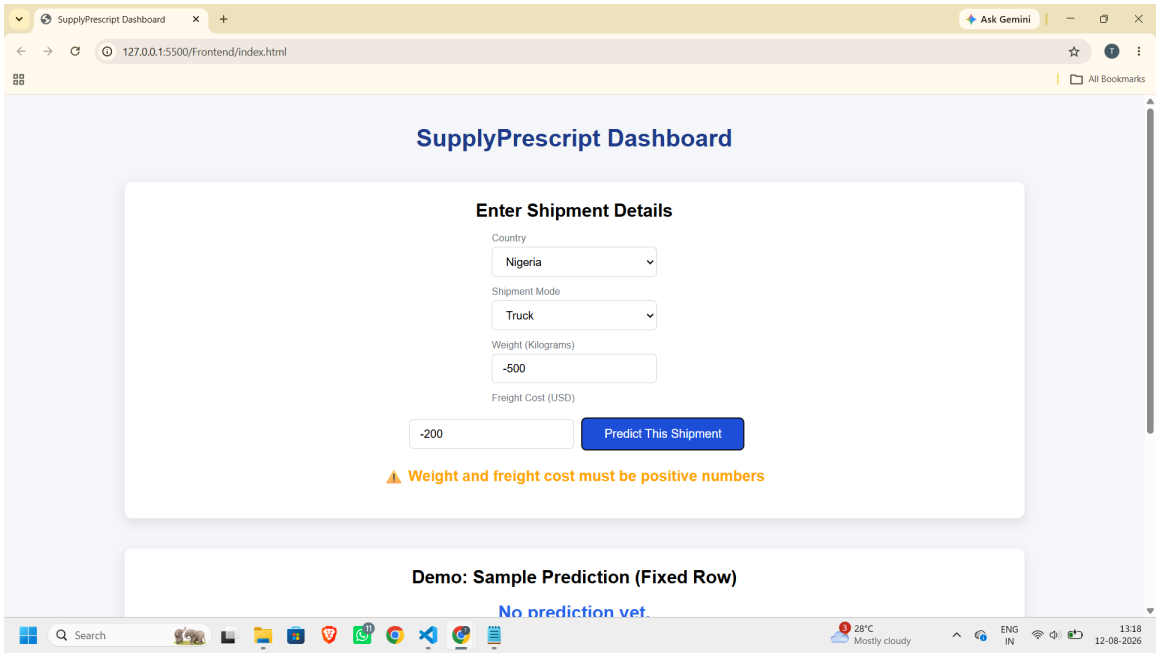

Figure 11 — Input validation: negative numbers correctly blocked

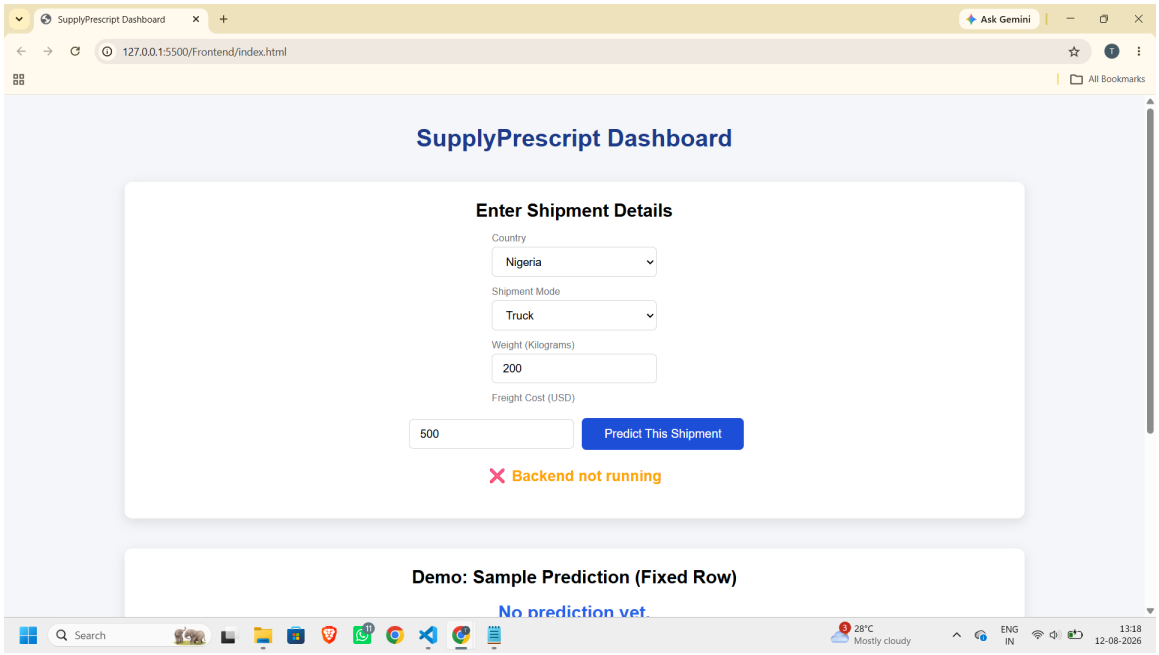

Figure 12 — Graceful error handling when the backend is offline

7. Application Testing

- Empty fields → blocked with a clear warning.
- Negative or zero numbers → blocked with a validation warning (Figure 11).
- Extremely large numbers → handled without crashing.
- Backend down → graceful "Backend not running" message, no freeze (Figure 12).

- Missing fields via direct API call → clean JSON error, no raw crash page.
- Unknown category values (e.g. an invalid country) → gracefully falls back to defaults instead of crashing.

8. Known Limitations

- **/predict_custom** exposes only 4 of 157 model features; remaining columns use training-data defaults, which may not reflect a specific shipment's true characteristics.
- No authentication on any endpoint.
- CORS is hardcoded to one origin — must be updated if the frontend moves to a different port or domain.
- Model recall is 0.34 (Figure 3), meaning it misses roughly two-thirds of actual delayed shipments. This reflects the real difficulty of predicting a rare event (~11.5% delay rate) with the available features, not a bug.
- The backend does not independently re-validate that weight/freight cost are positive — it relies on frontend validation.
- Recommended Actions use simplified, transparent multiplier-based logic tied to real input, not a true constrained optimization engine (e.g. SciPy/PuLP linear programming) as envisioned in the original project specification. A full prescriptive, write-back, closed-loop learning system (matching real outcomes back against predictions to retrain the model) remains a planned future enhancement.

9. Conclusion & Recommendations

SupplyPrescript is a complete, working, end-to-end machine learning product — not just notebooks. Every stage, from data cleaning through live deployment, was independently verified with real evidence (screenshots, live tests, metric comparisons) rather than assumed.

Recommended next steps:

- Implement a true constrained optimization engine (SciPy/PuLP) for Recommended Actions, subject to real business constraints such as budget and time.
- Add a lightweight database (e.g. SQLite or PostgreSQL) to log chosen actions and their real outcomes, enabling closed-loop model retraining.
- Expand /predict_custom to accept more of the 157 model features directly from the user for higher-fidelity predictions.
- Add authentication to the API before any production or multi-user deployment.

— End of Report —